Professional Case Study

Name: Agnik Dutta
Role Applied: Full Stack Engineering Intern
Project: HR Workflow Designer
GitHub Repo Link : https://github.com/agnik07/HR-Workflow-Designer
Live Demo Link : https://hr-workflow-designer-beige.vercel.app/

HR Workflow Designer - Case Study Submission

1. Executive Summary

I created an interactive HR Workflow Designer where HR admins can create, configure, validate, and even simulate their workflows for onboarding, approval of leaves, document verification, etc.

The system is built using an interactive workflow canvas based on React Flow, along with configurable forms within the nodes, workflow validations, simulation logs, and even persistence capabilities.

As per the brief requirements, a frontend mock API prototype had to be delivered; however, I took the opportunity to build a complete stack MERN application to showcase production-level engineering skills.

2. Tech Stack

Layer	Technology
Frontend	React.js
Canvas Engine	React Flow
State Management	Zustand
Styling	Tailwind CSS + ShadCN UI
Backend	Node.js + Express
Database	MongoDB
API Layer	Axios
Graph Layout	Dagre
Notifications	Sonner

3. Core Features Delivered

3.1 Workflow Canvas

a) Nodes dragged from the sidebar
b) Edges created between nodes
c) Node/Edge deletion capabilities

d) Node selection for editing its properties
e) Zoom/Mini-map/Controls support
f) Auto-layout support

3.3 Nodes Implemented

3.3.1 Start Node

a) Title
b) Key-value metadata fields

3.3.2 Task Node

a) Title
b) Description
c) Assignee
d) Due date
e) Custom metadata

3.3.3 Approval Node

a) Title
b) Approver role
c) Auto-approval threshold

3.3.4 Automated Task Node

a) Title
b) Action selected from the API
c) Dynamic metadata

3.3.5 End Node

a) End message
b) Summary toggles

3.4 Mock/Real API Layer

GET /automations

Returns a list of all available actions:

```
[
 { "id":"send_email", "label":"Send Email" },
 { "id":"generate_doc", "label":"Generate Document" }
]
```

POST /simulate

Takes a workflow in JSON format and returns its log of stepwise execution.

4. Workflow Sandbox / Simulation

The simulation window consists of:

a) Serializing graph data
b) Submitting the workflow for processing
c) Validation of the graph
d) Displaying execution logs chronologically

Example:

- Workflow began
- Docs were assigned
- Applying for approval by manager
- Sending email
- Workflow ended

5. Validation Rules

Performed on frontend and backend:

a) One start node only
b) End nodes at least 1
c) Negative feedback loop
d) Nodes that do not belong
e) Titles must be present

6. Project Structure Design

Project was divided into:

a) Canvas logic
b) Node components
c) Node forms
d) API services
e) Global store
f) Utility hooks

It allows project to be extended easily and add new nodes.

7. Extra Features Implemented

a) Undo & Redo
b) JSON import/export
c) Graph auto-layout
d) Themes support
e) Workflow save/load
f) MongoDB storage

g) Workflow library

8. Important Engineering Choices

a) Zustand instead of Redux
b) Lighter, faster, less boilerplate
c) React Flow
d) Best tooling for node-based graphs
e) MongoDB
f) Flexible schemas for node-based graph storage.

9. One Tricky Bug Solved

While implementing drag-and-drop, dropped nodes appeared offset from cursor position.

Root Cause:

Browser coordinates were viewport-relative while React Flow uses transformed canvas coordinates.

Fix:

Used project() helper with wrapper bounds to map coordinates accurately.

10. Future Improvements

- Conditional branching nodes
- Real-time collaboration
- Role-based access
- Version history
- Workflow analytics dashboard

11. Conclusion

This project demonstrates my ability to rapidly build scalable frontend systems, integrate backend APIs, design maintainable architecture, and solve real product problems.

I enjoyed building this workflow engine and would be excited to contribute similar zero-to-one products at Tredence.